

Introduction to PID Control

The proportional-integral-derivative controller, one term at a time.

Ogata, *Modern Control Engineering*, Ch. 8.

In this tutorial you will:

- see what each of the **P**, **I**, and **D** terms does on its own,
- combine them and watch steady-state error and overshoot improve, and
- build PID controllers as transfer functions with **tf** and **feedback**.

Step through with **Ctrl+Enter**, or render a report with **publish**.

A PID controller computes its actuating signal from the error $e(t) = r(t) - y(t)$ as a weighted sum of proportional, integral, and derivative terms:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

with transfer function (often written with $K_i = K_p/T_i$, $K_d = K_p T_d$):

$$G_c(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right)$$

where T_i is the **integral (reset) time** and T_d the **derivative (rate) time**.

Contents

- Plant Used Throughout This File
- Before: The Uncontrolled Plant
- Proportional (P) Control
- Proportional-Integral (PI) Control
- Proportional-Derivative (PD) Control
- Full PID Control
- Try it yourself

Plant Used Throughout This File

$G(s) = \frac{1}{(s+1)(s+2)}$, unity feedback, step reference.

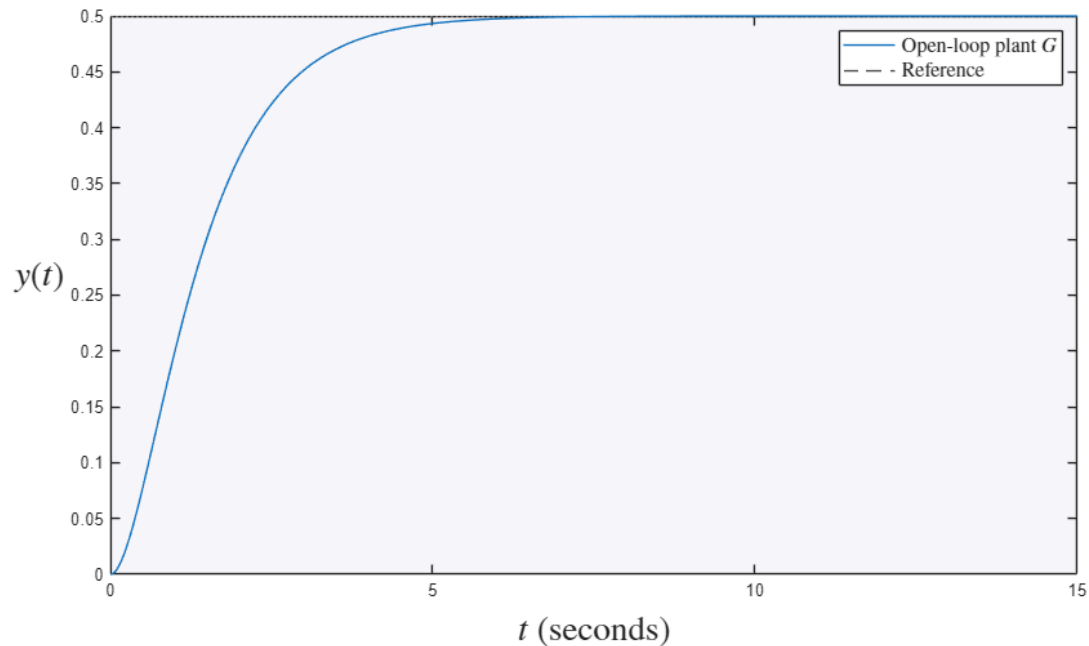
```
G = tf(1,conv([1 1],[1 2]));  
t = 0:0.01:15;
```

Before: The Uncontrolled Plant

Start from the bare plant's open-loop step response (no controller). It is sluggish and – being type-0 – settles well short of the reference, leaving a large steady-state error. Each PID term below attacks a different deficiency of this baseline.

```
figure  
step(G,t)  
hold on  
yline(1,'k--')  
hold off  
legend('Open-loop plant $G$', 'Reference', 'Interpreter', 'latex', 'FontSize', 12)  
title('Before: Uncontrolled Plant Step Response', 'Interpreter', 'latex', 'FontSize', 17)  
ylabel('$y(t)$', 'Interpreter', 'latex', 'FontSize', 20)  
set(get(gca, 'YLabel'), 'Rotation', 0)  
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)
```

Before: Uncontrolled Plant Step Response



Proportional (P) Control

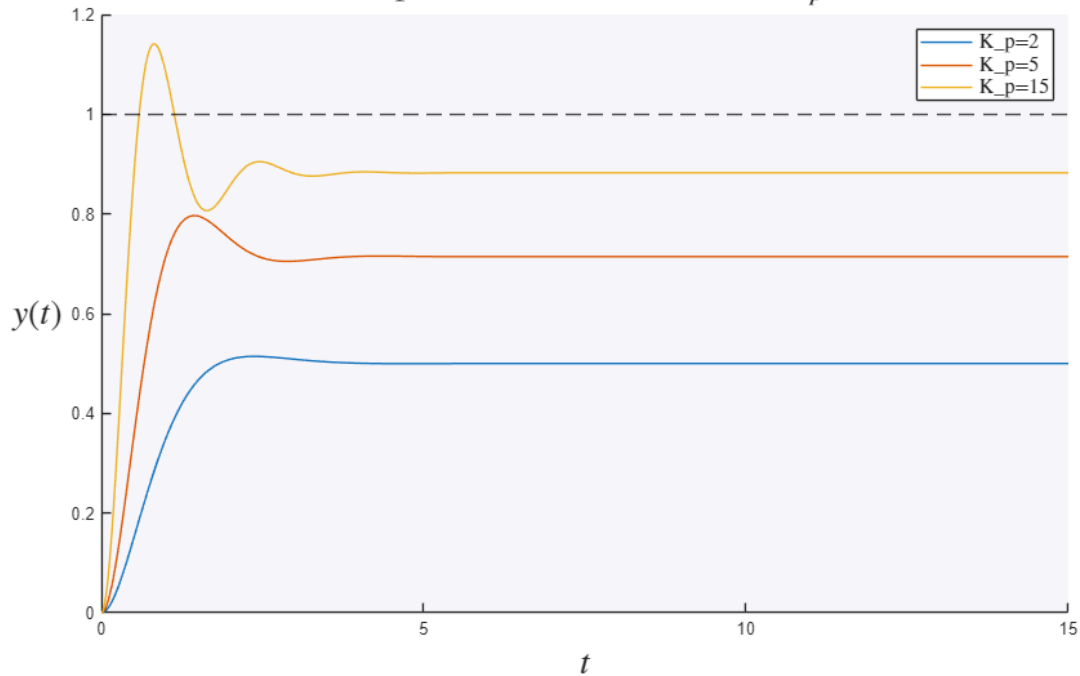
$G_c(s) = K_p$. Increasing K_p speeds the response and reduces (but does not eliminate) steady-state error for a type-0 plant, at the cost of increased overshoot.

```
Kps = [2 5 15];
figure
hold on
for Kp = Kps
    T = feedback(Kp*G,1);
    [y,tt] = step(T,t);
    plot(tt,y,'DisplayName',sprintf('K_p=%d',Kp))
end
yline(1,'k--','HandleVisibility','off')
hold off
legend('Interpreter','latex','FontSize',12)
title('Proportional Control: Effect of  $K_p$ ','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$','Interpreter','latex','FontSize',20)

Kp_demo = 5;
T_p = feedback(Kp_demo*G,1);
ess_p = 1 - dcgain(T_p);
fprintf('P control (Kp=%d): steady-state error = %.4f\n', Kp_demo, ess_p)
```

```
P control (Kp=5): steady-state error = 0.2857
```

Proportional Control: Effect of K_p



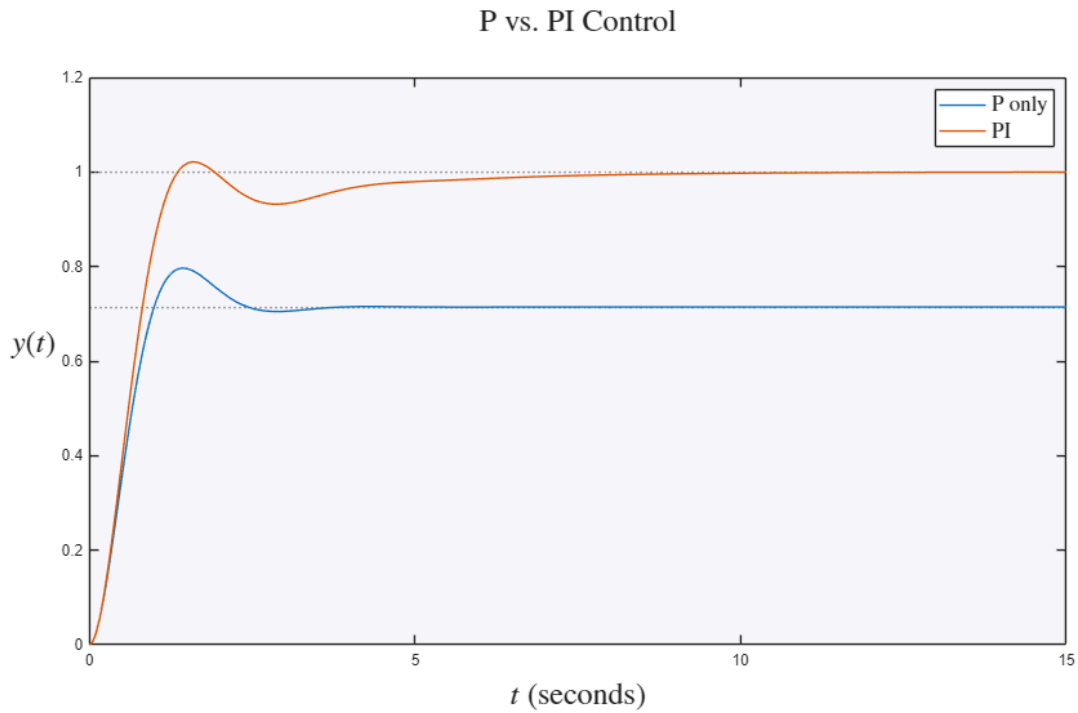
Proportional-Integral (PI) Control

$G_c(s) = K_p \left(1 + \frac{1}{T_i s}\right)$. The integral term adds a pole at the origin, making the loop type-1 and driving steady-state step error to zero, at the cost of a slower response and reduced phase margin (more oscillatory) compared to P alone.

```
Kp_pi = 5; Ti = 2;
Gc_pi = tf(Kp_pi*[1 1/Ti],[1 0]);
T_pi = feedback(Gc_pi*G,1);
ess_pi = 1 - dcgain(T_pi);
fprintf('PI control: steady-state error = %.6f (should be ~0)\n', ess_pi)
```

```
figure
hold on
step(T_p,t)
step(T_pi,t)
hold off
legend('P only','PI','Interpreter','latex','FontSize',14)
title('P vs. PI Control','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```

```
PI control: steady-state error = 0.000000 (should be ~0)
```



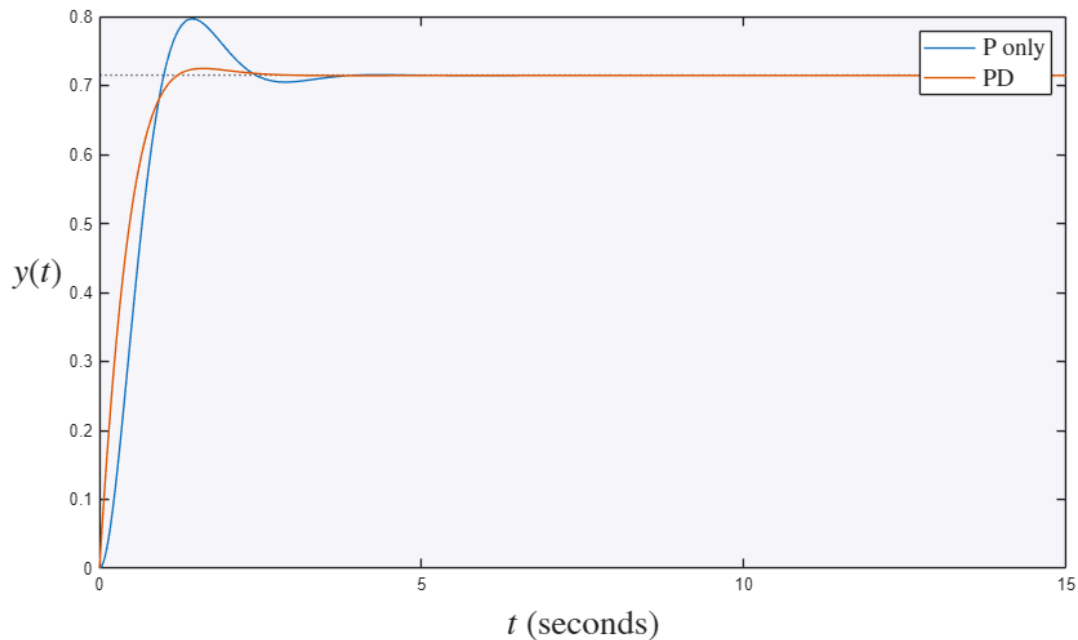
Proportional-Derivative (PD) Control

$G_c(s) = K_p(1 + T_d s)$. The derivative term adds phase lead, improving damping/reducing overshoot and speeding settling, but does not affect steady-state error and amplifies high-frequency measurement noise.

```
Kp_pd = 5; Td = 0.3;
Gc_pd = tf(Kp_pd*[Td 1],1);
T_pd = feedback(Gc_pd*G,1);

figure
hold on
step(T_p,t)
step(T_pd,t)
hold off
legend('P only','PD','Interpreter','latex','FontSize',14)
title('P vs. PD Control','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```

P vs. PD Control



Full PID Control

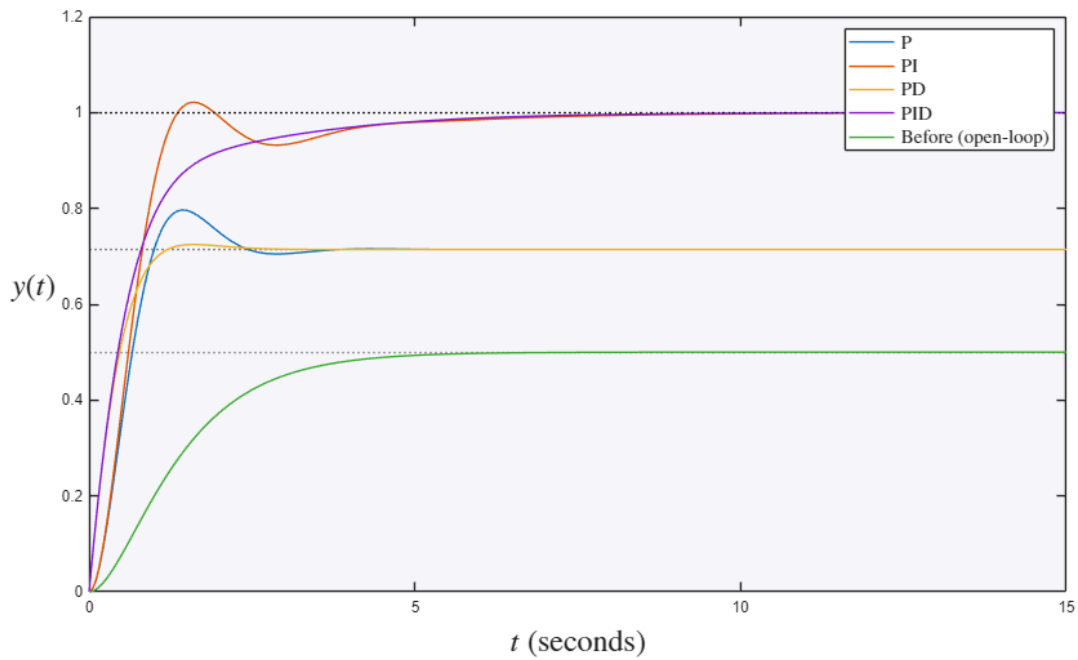
Combining all three terms gives zero steady-state error (from the integral action) together with good transient response (from the derivative action).

```
Kp = 5; Ti = 2; Td = 0.3;
% Gc(s) = Kp*(Td*s + 1 + 1/(Ti*s)) = Kp*(Td*s^2 + s + 1/Ti)/s
Gc_pid = tf(Kp*[Td 1 1/Ti],[1 0]);
T_pid = feedback(Gc_pid*G,1);
ess_pid = 1 - dcgain(T_pid);
fprintf('PID control: steady-state error = %.6f\n', ess_pid)

figure
hold on
step(T_p,t)
step(T_pi,t)
step(T_pd,t)
step(T_pid,t)
step(G,t)
yline(1,'k:','HandleVisibility','off')
hold off
legend('P','PI','PD','PID','Before (open-loop)','Interpreter','latex','FontSize',13)
title('After: P, PI, PD, PID vs. the Uncontrolled Baseline','Interpreter','latex','FontSize',16)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```

```
PID control: steady-state error = 0.000000
```

After: P, PI, PD, PID vs. the Uncontrolled Baseline



Try it yourself

- Zero out the integral term ($T_i = \text{inf}$ in the PID) and watch the steady-state error return – integral action is what removes it.
- Push T_d up and notice the overshoot fall but the response get twitchier (derivative amplifies fast changes, including noise).